COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structures And Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	09/18/26	**DATE SUBMITTED:**	09/1826
NAME:	Acala, Matt John C.	**INSTRUCTOR:**	Engr. Jimlord M.Quejado

HOA 9.1 Binary Search Tree

1. PROCEDURE OUTPUT

Task 1: Create code in C++ that will create a tree as shown in the figure above. Use linked lists as the internal representation of this tree. Indicate your code screenshot and comments in table 9-1.
Task 2: Complete the following table:

Node	Height	Depth
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
P	0	3
Q	0	3

Task 3: After implementing the code for above, answer the following:
3.1 Given the tree diagram, find the result of the pre-order, post-order and in-order traversal strategies by hand. Include this as table 9-3 in section
3.2 Create a function for pre-order, post-order and in-order traversal. Make sure that each function displays an output into the console. Once you have the output, create and fill table 9-4 in section 6 so that it contains the screenshot of the function, screenshot of the output and your observations. Your observations consist of a comparison between output in #1 and the output of your functions in #2.

```
template <typename T>
void Tree<T>::preOrder() {
    std::cout << key << " ";
    if (leftChild != nullptr) {
        leftChild->preOrder();
    }
    if (rightChild != nullptr) {
        rightChild->preOrder();
    }
}

template <typename T>
void Tree<T>::inOrder() {
    if (leftChild != nullptr) {
        leftChild->inOrder();
    }
    std::cout << key << " ";
    if (rightChild != nullptr) {
        rightChild->inOrder();
    }
}

template <typename T>
void Tree<T>::postOrder() {
    if (leftChild != nullptr) {
        leftChild->postOrder();
    }
    std::cout << key << " ";
    if (rightChild != nullptr) {
        rightChild->postOrder();
    }
}
```

3.3. Create a function called findData that will take the following parameter: “CHOICE, KEY”. Choice will determine which traversal method will be used to traverse the tree to find the second parameter KEY. If the KEY

is found, the value will be displayed such that: “{KEY} was found!”. Otherwise, no output. Include your output as table 9-5 in section 6.

```
template <typename T>
bool Tree<T>::findData(T value, int choice) {
    if (choice == 1) { // Preorder
        if (key == value) {
            std::cout << value << " was found!" << std::endl;
            return true;
        }
        if (leftChild != nullptr && leftChild->findData(value, choice)) return true;
        if (rightChild != nullptr && rightChild->findData(value, choice)) return true;
    }
    else if (choice == 2) { // Inorder
        if (leftChild != nullptr && leftChild->findData(value, choice)) return true;
        if (key == value) {
            std::cout << value << " was found!" << std::endl;
            return true;
        }
        if (rightChild != nullptr && rightChild->findData(value, choice)) return true;
    }
    else if (choice == 3) { // Postorder
        if (leftChild != nullptr && leftChild->findData(value, choice)) return true;
        if (key == value) {
            std::cout << value << " was found!" << std::endl;
            return true;
        }
        if (rightChild != nullptr && rightChild->findData(value, choice)) return true;
    }
    return false;
```

3.4. Add a new leaf node called which will contain the data value of “O”. Make this node the child of nodeG. Perform the findData function you created in #3, what can you observe about the output? Include your output and answers as table 9-6 in section 6.

Pre-order	Preorder: A B C D H E I J P Q F K L M G N After adding O: Preorder: A B C D H E I J P Q F K L M G N O
In-order	Inorder: B C H D I P Q J E K L M F N G A Inorder: B C H D I P Q J E K L M F N O G A
Post-order	Postorder: B C H D I P Q J E K L M F N G A Postorder: B C H D I P Q J E K L M F N O G A

Adding node "O" as a child of "G" makes it a sibling to "N". The findData function successfully locates it and prints "O was found!". This shows that using a linked list makes adding new nodes easy and flexible.

```
Preorder: A B C D H E I J P Q F K L M G N
Inorder: B C H D I P Q J E K L M F N G A
Postorder: B C H D I P Q J E K L M F N G A

E was found!
G was found!

After adding O:
Preorder: A B C D H E I J P Q F K L M G N O
Inorder: B C H D I P Q J E K L M F N O G A
Postorder: B C H D I P Q J E K L M F N O G A

O was found!
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

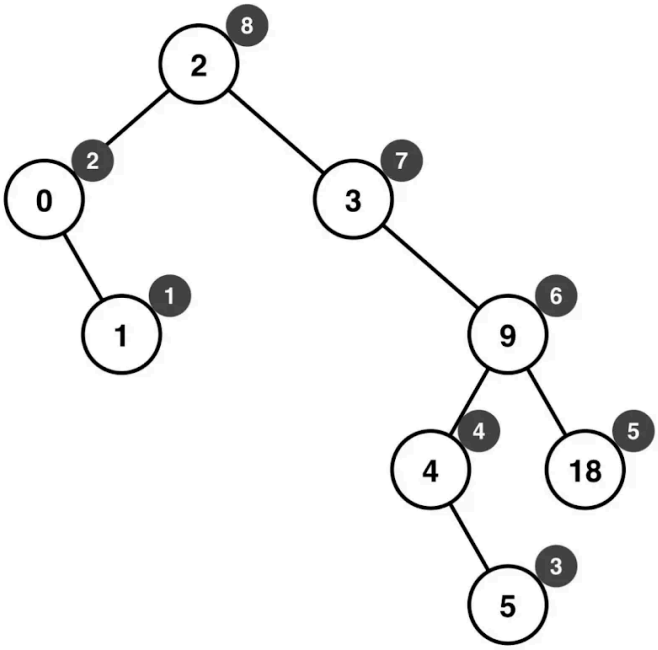
Step 1: Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.

```
// create the root node
BinarySearchTree<int> tree(2);

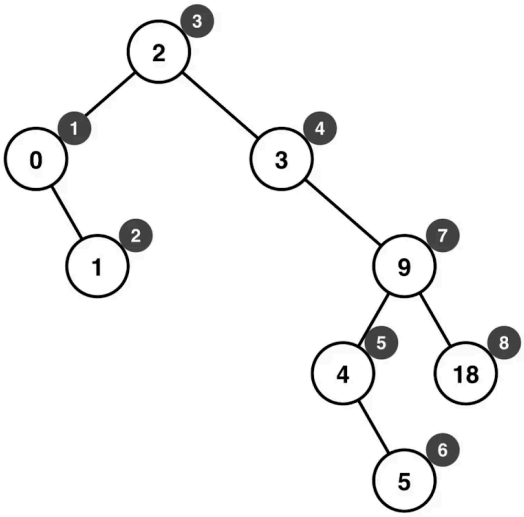
tree.insert(3);
tree.insert(9);
tree.insert(18);
tree.insert(0);
tree.insert(1);
tree.insert(4);
tree.insert(5);
```

Step 2: Create a diagram to show the tree after all values have been inserted. Then, with the use of visual aids (like arrows and numbers) indicate the traversal order for in-order, pre-order and post-order traversal on the diagram

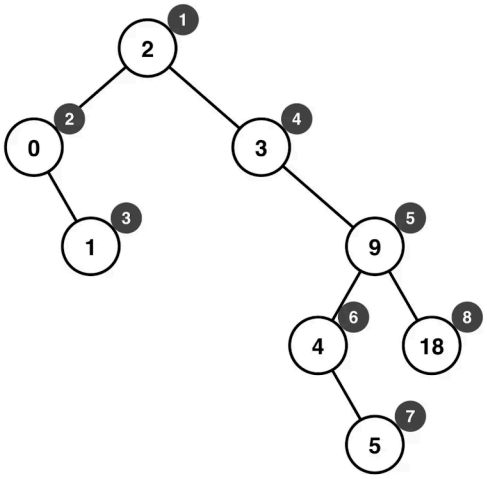
Post-order Traversal



In-order Traversal



Pre-order Traversal



Step 3: Compare the different traversal methods. In-order traversal was performed with what function?

```
Preorder: 2 0 1 3 9 4 5 18
Inorder: 0 1 2 3 4 5 9 18
Postorder: 1 0 5 4 18 9 3 2
```

Given the same input values above, what is the output with different traversal methods? For each output below, indicate your observation: is the output different from the pre-order and post-order traversal that you indicated in the diagrams shown in item #2

The in-order output is always sorted

The pre-order output differs from in-order because it emphasizes the root-first structure.

The post-order output is different again, because it emphasizes children-first traversal.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **CoPilot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Makes me digest easily hard concepts**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.